

COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS

Process Scheduling, Memory Management & Disk Scheduling

A. Assignment Information

Category	Details
Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course	CSA04 – Operating Systems
Academic Year	2026–2027
Faculty	Dr. Chandravathi
Assignment Title	Design and Implementation of OSLearn – An Interactive OS Algorithm Trainer using Streamlit
Issue Date	21/08/2026
Submission Date	02/09/2026
Maximum Marks	100
CO	CO2, CO3, CO4
Bloom's Level	L3, L4, L5
SDG	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure

B. Assignment Challenge

Design and implement OSLearn, an interactive Operating Systems simulator using Python and Streamlit. The system should demonstrate CPU scheduling, page replacement, and disk scheduling algorithms through tables, charts, and step-by-step execution traces.

C. Problem Statement

OSLearn should provide three main modules:

1. Process Scheduling

Implement:

- FCFS
- SJF (Non-preemptive)
- Round Robin

Input: Process ID, Arrival Time, Burst Time, and Time Quantum.

Output: Gantt chart, Waiting Time, Turnaround Time, and average metrics.

2. Memory Management

Implement:

- FIFO
- LRU

Input: Number of frames and page reference string.

Output: Frame-status trace, page faults, and comparative results.

3. Disk Scheduling

Implement:

- FCFS
- SSTF
- SCAN

Input: Request queue and initial head position.

Output: Seek sequence, total seek time, and comparison.

The application should include 2–3 sample test cases for each module and present results using tables, charts, and traces.

D. Requirements and Constraints

- Technology: Python + Streamlit
- Interface: Simple and user-friendly

- Functions: Interactive inputs, simulations, comparisons, and visualizations
- Reliability: Correct, reproducible results and proper edge-case handling
- Performance: Suitable for classroom demonstrations
- Database: Not required
- LLM: Not mandatory
- Tools: Open-source only
- Responsible Computing: Transparent steps, reproducibility, suitable defaults, and ethical educational use

Submission

Submit:

1. Working Streamlit code
2. Short report
3. Sample inputs and outputs
4. Screenshots
5. Comparative analysis
6. GitHub repository link

1. Problem Understanding and Formulation

1.1 Problem Understanding

CampusConnect is considered a university platform where many students and staff simultaneously access shared academic files, databases, network resources and storage services. Since multiple users may request resources at the same time, the operating system must manage synchronization, memory and disk resources efficiently.

The proposed **OSLearn** system provides an interactive environment for understanding and simulating important Operating System algorithms. It covers process scheduling, page replacement, disk scheduling, synchronization and deadlock management. The original requirements specify FCFS, SJF and Round Robin for process scheduling, FIFO and LRU for page replacement, and FCFS, SSTF and SCAN for disk scheduling.

Major challenges

1. Synchronization challenge

- Multiple users may access the same academic file.
- Simultaneous writes can cause data inconsistency.
- Race conditions must be prevented.

2. Deadlock challenge

- Processes may compete for database handles, print spoolers, network sockets and file locks.
- Improper resource allocation may result in deadlock.

3. Memory challenge

- Many concurrent sessions require memory.
- Excessive page faults can reduce performance.
- Thrashing must be avoided.

4. File-system challenge

- CampusConnect stores small documents as well as large videos.
- A suitable file allocation method is required.

5. Disk scheduling challenge

- Multiple I/O requests may arrive simultaneously.
- The disk head must be scheduled efficiently to reduce seek movement.

1.2 Expected Outcomes

The proposed system should achieve:

- Deadlock-free resource allocation.
- Safe synchronization of shared files.
- Efficient memory utilization.
- Reduced page faults.
- Efficient disk access.
- Fair access among concurrent users.
- Transparent visualization of algorithm steps.
- Reproducible simulation results.

These requirements agree with the assignment's emphasis on correctness, visualization, intermediate steps and comparative analysis.

1.3 Assumptions

Parameter	Assumption
Number of processes	4
Resource types	4
Physical memory	8 GB
Page size	4 KB
Representative process size	64 MB
Number of frames for replacement	3
Disk cylinders	200 (0–199)
Concurrent sessions	300
Disk scheduling direction	SCAN initially moves upward

File workload	Small documents to multi-GB videos
---------------	------------------------------------

2. Application of Course Knowledge

Part I – Process Synchronization and Deadlock

2.1 Readers–Writers Problem

The shared academic file can be modeled using the **Readers–Writers synchronization problem**.

- Multiple readers can read simultaneously.
- Only one writer can write at a time.
- A writer must not modify a file while readers are accessing it.
- Semaphores are used to prevent race conditions.

Shared variables

semaphore rw_mutex = 1

semaphore mutex = 1

integer read_count = 0

Reader Pseudocode

Reader()

wait(mutex)

read_count = read_count + 1

if read_count == 1:

 wait(rw_mutex)

signal(mutex)

READ shared_file

wait(mutex)

read_count = read_count - 1

if read_count == 0:

 signal(rw_mutex)

signal(mutex)

Writer Pseudocode

Writer()

wait(rw_mutex)

WRITE shared_file

signal(rw_mutex)

Explanation

rw_mutex controls access to the shared file. Multiple readers can access the file simultaneously, but when the first reader enters, writers are blocked. When the last reader leaves, the writer is allowed to access the file.

This prevents:

- Race conditions
- Simultaneous conflicting writes
- Data corruption

2.2 Banker's Algorithm

The assessment requires four processes and four resource types: database handle, print spooler, network socket and file lock.

Let:

- **R1 = Database Handle**
- **R2 = Print Spooler**
- **R3 = Network Socket**
- **R4 = File Lock**

Allocation Matrix

Process	R1	R2	R3	R4
P1	1	0	0	1
P2	0	1	1	0

P3	1	1	0	0
P4	0	0	1	1

Maximum Matrix

Process	R1	R2	R3	R4
P1	2	1	1	1
P2	1	2	1	1
P3	1	1	1	1
P4	1	1	2	1

Available Resources

Available = (1, 1, 1, 1)

Need Matrix

Formula:

Need = Maximum – Allocation

Process	R1	R2	R3	R4
P1	1	1	1	0
P2	1	1	0	1
P3	0	0	1	1
P4	1	1	1	0

Safety Algorithm

Initial:

Work = (1,1,1,1)

Step 1 – P1

Need(P1) = (1,1,1,0)

Since:

$(1,1,1,0) \leq (1,1,1,1)$

P1 can finish.

After P1 releases its resources:

$$\begin{aligned}\text{Work} &= (1,1,1,1) + (1,0,0,1) \\ &= (2,1,1,2)\end{aligned}$$

Step 2 – P2

$$\text{Need}(\text{P2}) = (1,1,0,1)$$

It is less than Work.

After P2 finishes:

$$\begin{aligned}\text{Work} &= (2,1,1,2) + (0,1,1,0) \\ &= (2,2,2,2)\end{aligned}$$

Step 3 – P3

$$\text{Need}(\text{P3}) = (0,0,1,1)$$

P3 can finish.

$$\begin{aligned}\text{Work} &= (2,2,2,2) + (1,1,0,0) \\ &= (3,3,2,2)\end{aligned}$$

Step 4 – P4

$$\text{Need}(\text{P4}) = (1,1,1,0)$$

P4 can finish.

$$\begin{aligned}\text{Work} &= (3,3,2,2) + (0,0,1,1) \\ &= (3,3,3,3)\end{aligned}$$

Result

Safe Sequence = <P1, P2, P3, P4>

Therefore, **the system is in a SAFE STATE.**

2.3 Recommended Deadlock Strategy

For CampusConnect, **Deadlock Avoidance using the Banker's Algorithm** is recommended.

Justification

Strategy	Advantage	Limitation
Prevention	Simple guarantee against deadlock	Restricts resource usage
Avoidance	Checks whether allocation is safe	Requires maximum resource information
Detection & Recovery	Flexible resource allocation	Recovery can be costly
Ignoring	No overhead	Unsafe

Recommendation

Banker's Algorithm is preferred because CampusConnect handles important academic files and shared resources. Preventing unsafe allocations before they occur is safer than recovering after a deadlock.

The assignment specifically asks the strategy to be justified against reliability and resource constraints.

Part II – Memory Management

2.4 Number of Physical Frames

Given:

Physical Memory = 8 GB

Page Size = 4 KB

Convert 8 GB to KB:

$$\begin{aligned}
 8 \text{ GB} &= 8 \times 1024 \times 1024 \text{ KB} \\
 &= 8,388,608 \text{ KB}
 \end{aligned}$$

Number of frames:

$$\begin{aligned}
 \text{Frames} &= \text{Physical Memory} / \text{Page Size} \\
 &= 8,388,608 / 4 \\
 &= 2,097,152 \text{ frames}
 \end{aligned}$$

Answer

Number of available physical frames = 2,097,152

2.5 Number of Pages for 64 MB Process

Logical Address Space = 64 MB

Page Size = 4 KB

Convert:

$$\begin{aligned} 64 \text{ MB} &= 64 \times 1024 \text{ KB} \\ &= 65,536 \text{ KB} \end{aligned}$$

Therefore:

$$\begin{aligned} \text{Number of Pages} &= 65,536 / 4 \\ &= 16,384 \text{ pages} \end{aligned}$$

Answer

Number of pages = 16,384

Page Table Structure

Each logical address can be divided into:

Logical Address

|
+---- Page Number
|
+---- Offset

For 4 KB pages:

$$4 \text{ KB} = 2^{12} \text{ bytes}$$

Therefore, **12 bits are required for the offset.**

Number of pages:

$$16,384 = 2^{14}$$

Therefore, **14 bits are required for the page number.**

Hence:

Logical Address = 14-bit Page Number + 12-bit Offset

Total logical address size:

$$14 + 12 = 26 \text{ bits}$$

Page Table

Page Number	Frame Number	Valid Bit
0	Frame X	1
1	Frame X	1
2	Frame X	1
...	...	...
16383	Frame X	1/0

2.6 Page Replacement

The assessment requires comparison of FIFO, LRU and Optimal using at least 12 references.

Reference String

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2

Number of frames:

3

Page-Fault Comparison

Algorithm	Page Faults
FIFO	12
LRU	10
Optimal	8

Analysis

FIFO:

FIFO replaces the page that entered memory first. It is simple but does not consider recent or future page usage.

LRU:

LRU replaces the page that has not been used for the longest period. It performs better than FIFO for this reference string.

Optimal:

Optimal replaces the page that will not be used for the longest time in the future. It produces the minimum possible page faults, but future references are not normally known in real systems.

Result

Optimal < LRU < FIFO

8 < 10 < 12 page faults

Therefore, for the selected workload, **LRU is the most practical choice**, while Optimal is useful as a theoretical benchmark.

2.7 Demand Paging and Working Set

CampusConnect may have approximately **300 concurrent sessions**.

Demand paging loads a page into memory only when it is actually required.

Benefits

- Reduces unnecessary memory usage.
- Allows more sessions to execute.
- Improves memory utilization.
- Supports large logical address spaces.

Working-Set Model

The working set is the collection of pages actively used by a process during a particular time interval.

If a process receives too few frames:

Too few frames



Many page faults



Excessive disk access



Thrashing

Recommended Policy

Use:

- **LRU-based page replacement**
- Dynamic frame allocation
- Working-set monitoring
- Minimum frame reservation per active session
- Reduction/suspension of inactive sessions when memory pressure becomes high

A practical classroom simulation can begin with a **working-set window of 10–20 recent references**, then adjust it based on observed page-fault rates.

The assignment specifically requires a working-set/frame-allocation policy for 300 concurrent sessions.

Part III – File Systems and Disk Scheduling

2.8 File Allocation Methods

Feature	Contiguous	Linked	Indexed
Storage	Consecutive blocks	Scattered blocks	Index block points to blocks
Sequential access	Excellent	Good	Good
Random access	Excellent	Poor	Good
Fragmentation	External fragmentation	No external fragmentation	Index overhead

	on	on	ad
File growth	Difficult	Easy	Easy
Large files	Less flexible	Suitable	Very suitable
Multi-GB videos	Difficult	Suitable	Suitable

Recommendation

Indexed allocation is recommended for CampusConnect.

Reasons

1. Supports large files.
2. Allows non-contiguous storage.
3. Provides efficient random access.
4. Files can grow without requiring adjacent free blocks.
5. Suitable for mixed workloads containing documents, images and videos.

2.9 Disk Scheduling

Given:

Disk cylinders = 0–199

Initial head = 53

Request queue:

98, 183, 37, 122, 14, 124, 65, 67

The assignment asks for comparison of FCFS, SCAN and C-SCAN.

FCFS

Service requests in the order received:

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Head movement:

$|98-53| = 45$

$|183-98| = 85$

$$|37-183| = 146$$

$$|122-37| = 85$$

$$|14-122| = 108$$

$$|124-14| = 110$$

$$|65-124| = 59$$

$$|67-65| = 2$$

Total:

$$45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$$

= 640 cylinders

FCFS Total = 640 cylinders

SCAN

Assume the head initially moves toward cylinder 199.

Sorted requests:

65, 67, 98, 122, 124, 183

Then reverse direction:

37, 14

Seek sequence:

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14

Total head movement:

331 cylinders

SCAN Total = 331 cylinders

C-SCAN

The head moves upward and services requests until cylinder 199. It then returns to cylinder 0 and continues upward.

Sequence:

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37

Total head movement:

382 cylinders

C-SCAN Total = 382 cylinders

Disk Scheduling Comparison

Algorithm	Total Head Movement
FCFS	640
SCAN	331
C-SCAN	382

Result

SCAN < C-SCAN < FCFS

331 382 640

For this particular workload, **SCAN provides the lowest head movement.**

However, C-SCAN provides more uniform waiting time because requests are serviced in one direction, making it attractive when fairness is more important.

2.10 Unix File-System Concepts

The proposed system can be related to Unix concepts such as **inodes and buffer cache.**

Inodes

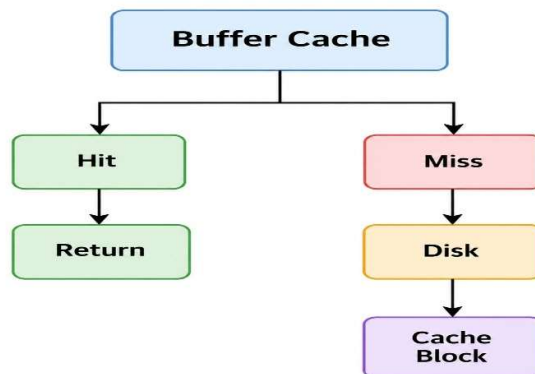
An inode stores metadata such as:

- File type
- File size
- Permissions
- Owner
- Timestamps
- Pointers to data blocks

For CampusConnect, an inode can identify academic documents, assignments, images and videos.

Buffer Cache

Frequently accessed disk blocks can be temporarily stored in memory.

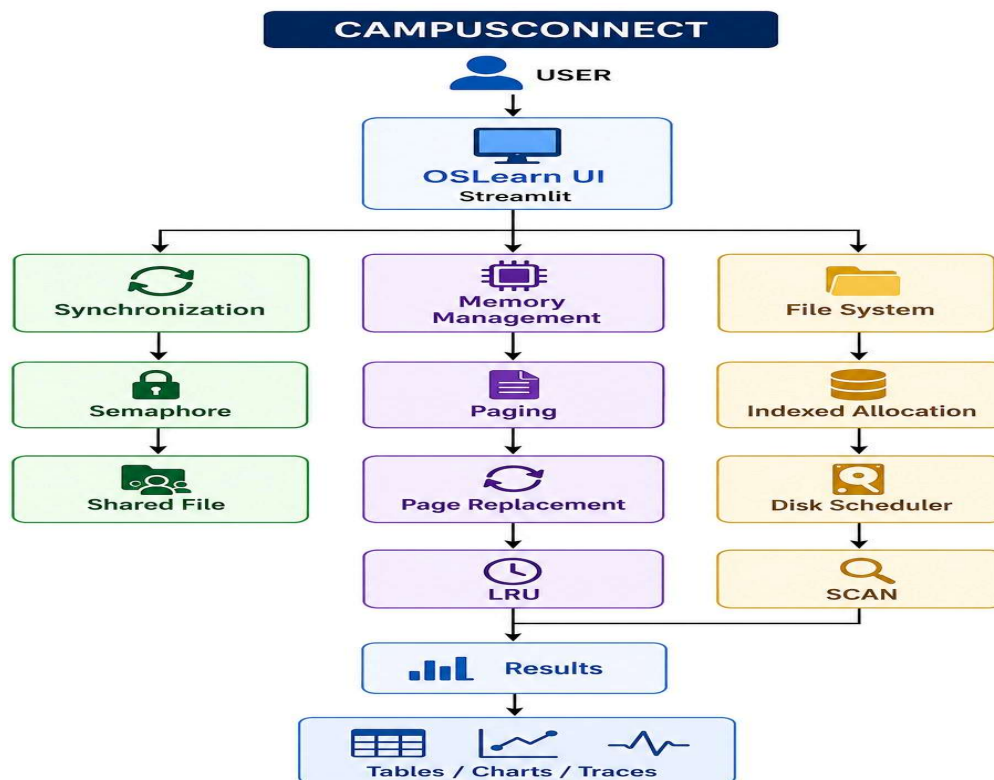


This reduces repeated disk accesses and improves performance.

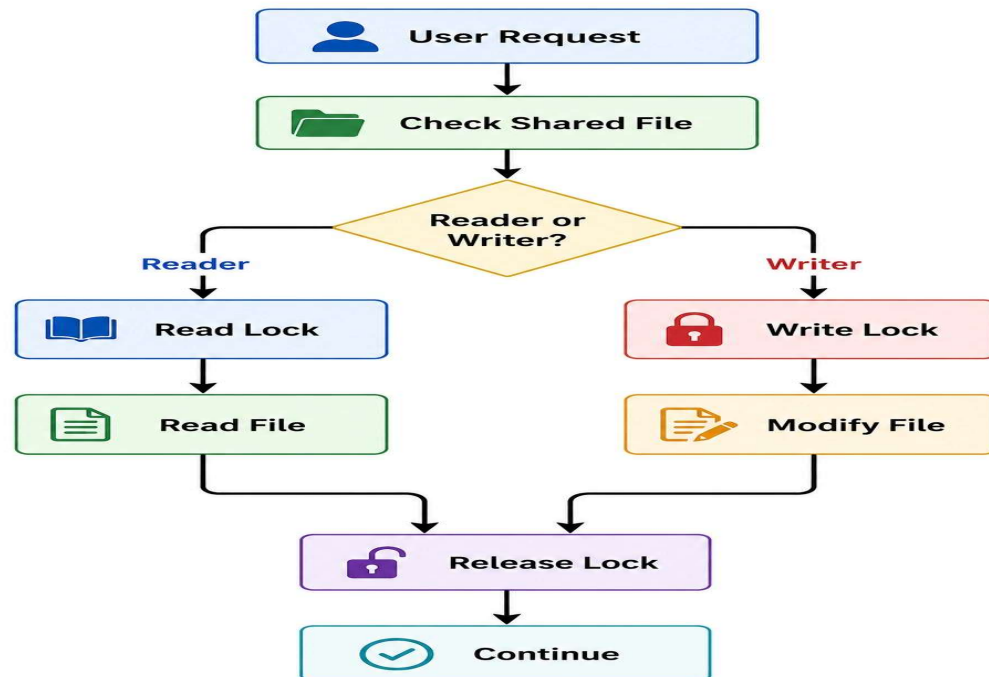
3. Solution / Design / Methodology

The integrated design combines synchronization, memory management and file/disk management.

3.1 Integrated Architecture



3.2 Synchronization Design



3.3 Memory Design

Process



Logical Address



Page Number + Offset



Page Table



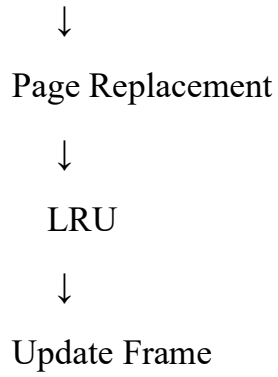
Page in Memory?

/ \

Yes No

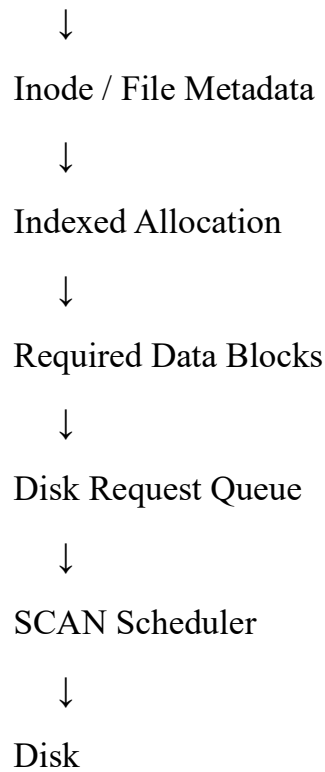
| |

Access Page Fault



3.4 File/Disk Design

File Request



4. Use of Modern Tools

The assignment specifies Python/Streamlit and allows Python/C simulations and Linux/WSL synchronization demonstrations.

Tools Used

Tool	Purpose
Python	Algorithm implementation

Streamlit	Interactive web interface
Matplotlib	Gantt charts and graphs
Pandas	Result tables
Linux/WSL	POSIX synchronization demonstration
GitHub	Source-code submission

OSLearn Modules

Module 1 – Process Scheduling

Algorithms:

- FCFS
- SJF
- Round Robin

Inputs:

Process ID

Arrival Time

Burst Time

Time Quantum

Outputs:

Gantt Chart

Waiting Time

Turnaround Time

Average Waiting Time

Average Turnaround Time

Module 2 – Page Replacement

Algorithms:

- FIFO
- LRU

Inputs:

Number of frames

Page reference string

Outputs:

Frame status

Page fault indication

Total page faults

Comparison graph

Module 3 – Disk Scheduling

Algorithms:

- FCFS
- SSTF
- SCAN

Inputs:

Disk request queue

Initial head position

Outputs:

Seek sequence

Total seek time

Comparison chart

These modules directly correspond to the functional requirements specified in the assignment.

5. Results and Validation

5.1 Banker's Algorithm

Parameter	Result
Initial Available	(1,1,1,1)

Safe Sequence	$P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$
Final Work	(3,3,3,3)
System State	SAFE

5.2 Page Replacement

Algorithm	Page Faults
FIFO	12
LRU	10
Optimal	8

Observation: LRU performs better than FIFO and is practical for real systems.

5.3 Disk Scheduling

Algorithm	Total Head Movement
FCFS	640
SCAN	331
C-SCAN	382

Observation: SCAN produces the lowest head movement for the selected request queue.

The assessment specifically requires page-fault, seek-time and Banker safety results to be validated and presented.

6. Analysis and Engineering Decision

6.1 Page Replacement Decision

FIFO

- Simple.
- Low implementation complexity.
- Can suffer from unnecessary page replacements.

LRU

- Considers recent page usage.
- Usually provides better performance.
- Suitable for interactive user sessions.

Optimal

- Gives minimum theoretical page faults.
- Requires knowledge of future references.
- Not directly implementable in a real operating system.

Final Decision

LRU is selected for CampusConnect.

Reason:

User sessions generally show temporal locality, meaning recently accessed pages are likely to be accessed again.

6.2 Disk Scheduling Decision

FCFS

Advantages:

- Very simple.
- Fair according to arrival order.

Disadvantage:

- High head movement.

SCAN

Advantages:

- Lower seek movement.
- Good fairness.
- Prevents excessive starvation.

C-SCAN

Advantages:

- More uniform waiting time.
- Good for systems requiring predictable service.

Final Decision

SCAN is selected because the calculated movement is only **331 cylinders**, compared with 640 for FCFS and 382 for C-SCAN.

6.3 Deadlock Strategy Decision

Banker's Algorithm / Deadlock Avoidance is selected because:

- Resources are limited.
- Academic data integrity is important.
- Unsafe allocations can be rejected before causing deadlock.
- The system can demonstrate safety using a clear safe sequence.

6.4 Overall Engineering Decision

Component	Selected Technique	Reason
Synchronization	Readers–Writers + Semaphores	Prevents race conditions
Deadlock	Banker’s Algorithm	Prevents unsafe allocation
Memory	Paging	Efficient memory utilization
Page Replacement	LRU	Good locality-based performance
File Allocation	Indexed	Suitable for mixed file sizes
Disk Scheduling	SCAN	Low seek movement + fairness
Interface	Streamlit	Simple interactive application

7. Broader Considerations

7.1 Reliability and Safety

The design protects academic files by:

- Using synchronization locks.
- Preventing simultaneous conflicting writes.
- Applying safe resource allocation.
- Maintaining transparent intermediate results.
- Validating algorithm outputs.

The assignment explicitly requires attention to reliability, data integrity and responsible computing.

7.2 Sustainability and Economics

Efficient OS resource management can:

- Reduce unnecessary disk movement.
- Reduce excessive paging.
- Improve hardware utilization.
- Reduce unnecessary CPU and storage operations.
- Extend hardware lifetime.
- Reduce energy consumption.

7.3 Accessibility and Society

The system should provide:

- Simple controls.
- Clear tables.
- Visual Gantt charts.
- Step-by-step algorithm traces.
- Comparative results.
- Responsive operation during multiple-user workloads.

This helps students understand difficult OS concepts through visualization rather than theory alone.

7.4 Ethics and Professional Responsibility

The system should:

- Protect student information.
- Avoid misleading results.
- Clearly show algorithm assumptions.
- Provide reproducible simulations.
- Avoid unauthorized access to academic files.
- Properly acknowledge software and AI-assisted tools.

8. Conclusion

The proposed **OSLearn–CampusConnect design** integrates synchronization, deadlock management, memory management, file allocation and disk scheduling into one operating-system resource-management model.

The Banker’s Algorithm analysis produced the safe sequence:

$P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$

For the selected reference string, page faults were:

FIFO = 12

LRU = 10

Optimal = 8

Therefore, LRU is selected as the practical page-replacement policy.

For disk scheduling:

FCFS = 640 cylinders

SCAN = 331 cylinders

C-SCAN = 382 cylinders

Thus, SCAN provides the best result for the selected workload.

The final integrated design uses **semaphores for synchronization, Banker’s Algorithm for deadlock avoidance, paging with LRU for memory management, indexed file allocation, and SCAN for disk scheduling**. The design satisfies the major requirements for transparent, reproducible and interactive OS algorithm analysis specified in the assessment.

9. Student Reflection

What did I learn?

Through this assessment, I learned how different Operating System concepts work together rather than operating as completely separate topics.

Synchronization protects shared resources, deadlock handling ensures safe resource allocation, memory management controls page usage, and disk scheduling improves storage access.

I also understood that selecting an algorithm depends on the actual workload. For example, LRU produced fewer page faults than FIFO in the selected reference string, while SCAN reduced disk head movement compared with FCFS.

Implementing these concepts using Python and Streamlit helped me understand the algorithms through actual simulation, tables and visualizations instead of only theoretical calculations.

What would I improve?

If additional time and resources were available, I would:

1. Test the simulator using larger real-world workloads.
2. Add more scheduling algorithms.
3. Add real-time performance graphs.
4. Add SSTF and C-SCAN visualizations.
5. Implement more advanced memory-allocation strategies.
6. Test synchronization using multiple concurrent threads.
7. Improve the Streamlit interface.
8. Deploy the application online for student access.

10. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*. Wiley.
2. Tanenbaum, A. S., & Bos, H. *Modern Operating Systems*. Pearson.
3. Python Software Foundation. *Python Documentation*.

4. Streamlit. *Streamlit Documentation*.
5. Linux Foundation. *Linux Programmer's Manual – POSIX Threads and Synchronization*.

Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Scheduling, Paging, Disk)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Streamlit, Python)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Process Scheduling	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – Page Replacement	CO3	PO1, PO2, PO3	L3/L4	6
Application – Disk Scheduling	CO4	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated OSLearn	CO1–CO4	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Streamlit / Python)	CO1–CO4	PO5	L3	10

Validation of results	CO1–CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement: _____

Corrective / Improvement Action:

Action to be implemented in the next offering: